Lab: Hashing Using OpenSSL

Estimated time: 45 minutes

Overview

This lab demonstrates the use of OpenSSL for cryptographic hashing, which is critical for verifying data integrity, securing passwords, and supporting digital signatures. Each command is broken down step-by-step to ensure clarity and understanding.

Learning objectives

By the end of this lab, you will be able to:

- Generate cryptographic hashes using OpenSSL
- Verify data integrity through hash comparison
- Use hashes for securing passwords and supporting digital signatures

Prerequisites (Optional)

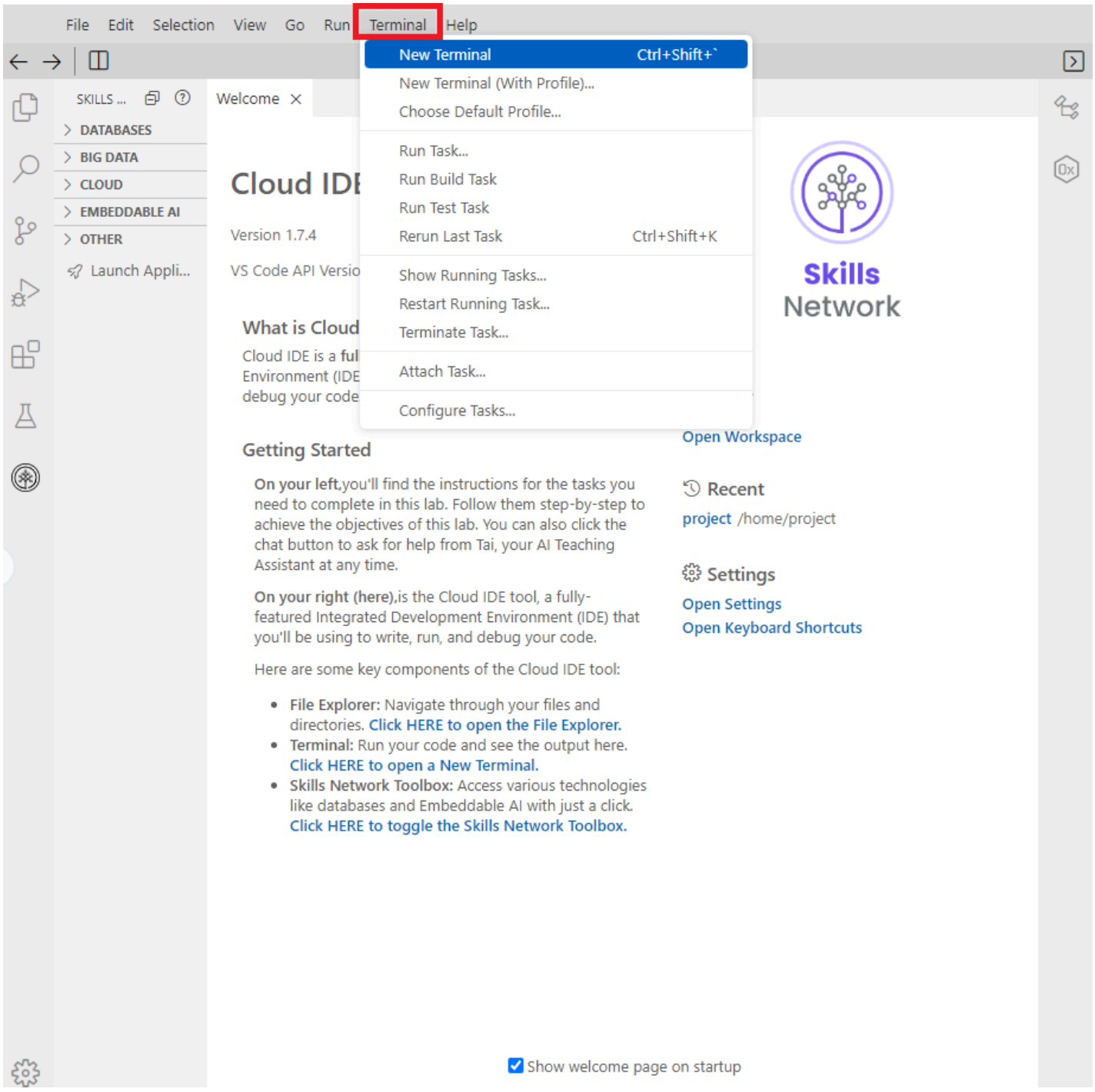
- Install OpenSSL on your system
- Basic understanding of the command line and hashing concepts is advantageous

---

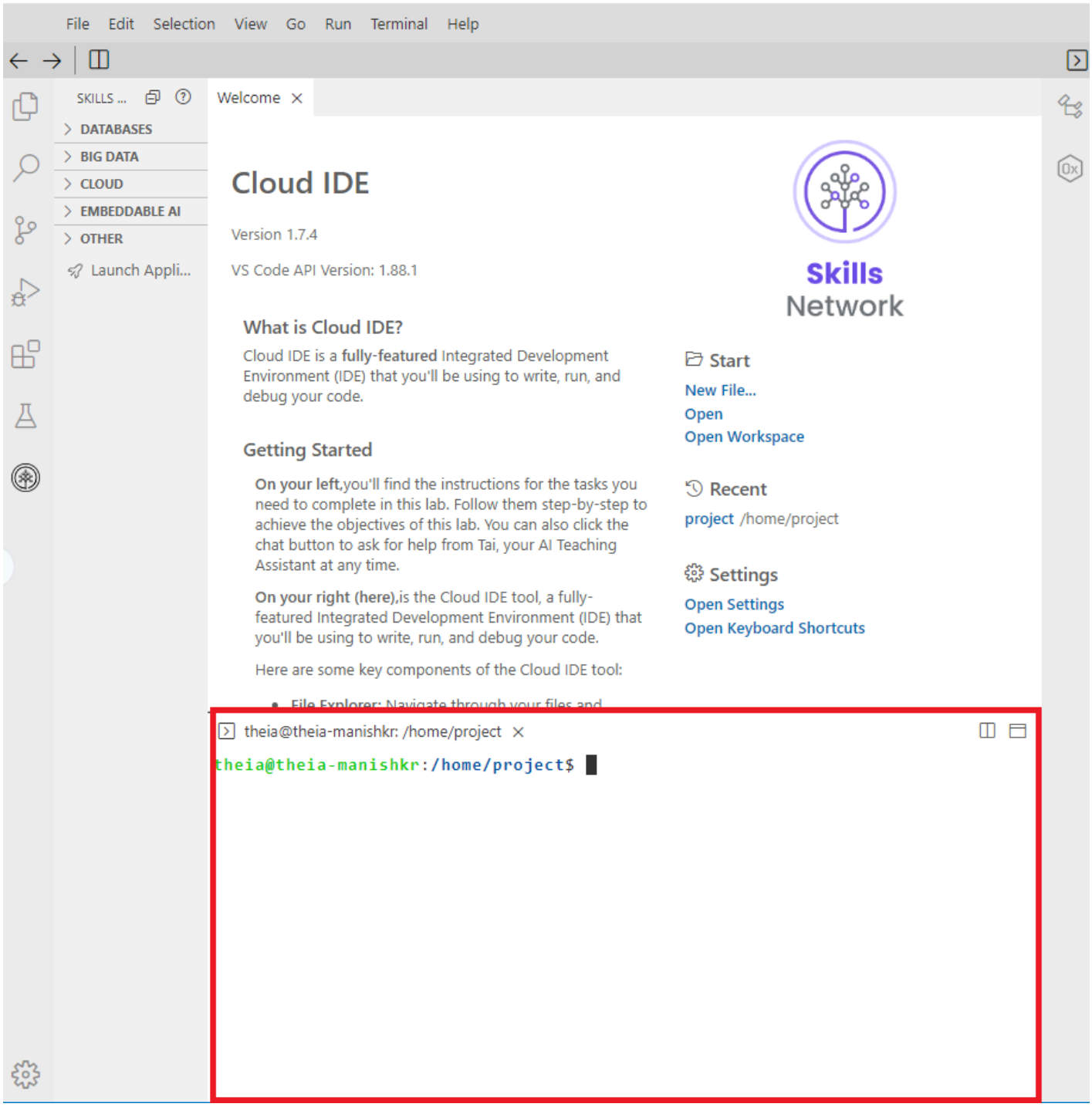
Initializing the Lab Environment

Follow the below steps to initialize the lab environment.

- Select **Terminal** and then select **New Terminal**. This will open a new terminal.


File Edit Selection View Go Run Terminal Help
New Terminal Ctrl+Shift+`
New Terminal (With Profile)...
Choose Default Profile...
Run Task...
Run Build Task
Run Test Task
Rerun Last Task Ctrl+Shift+K
Show Running Tasks...
Restart Running Task...
Terminate Task...
Attach Task...
Configure Tasks...
SKILLS ...
DATABASES
BIG DATA
CLOUD
EMBEDDABLE AI
OTHER
Launch Appli...
Welcome
Cloud IDE
Version 1.7.4
VS Code API Versio
What is Cloud
Cloud IDE is a ful
Environment (IDE
debug your code
Getting Started
On your left,you'll find the instructions for the tasks you need to complete in this lab. Follow them step-by-step to achieve the objectives of this lab. You can also click the chat button to ask for help from Tai, your AI Teaching Assistant at any time.
On your right (here),is the Cloud IDE tool, a fully-featured Integrated Development Environment (IDE) that you'll be using to write, run, and debug your code.
Here are some key components of the Cloud IDE tool:
File Explorer: Navigate through your files and directories. Click HERE to open the File Explorer.
Terminal: Run your code and see the output here. Click HERE to open a New Terminal.
Skills Network Toolbox: Access various technologies like databases and Embeddable AI with just a click. Click HERE to toggle the Skills Network Toolbox.
Skills Network
Open Workspace
Recent
project /home/project
Settings
Open Settings
Open Keyboard Shortcuts
Show welcome page on startup

- Copy and paste the commands into the terminal for the remaining steps.



Step 1: Verify OpenSSL Installation

Run the following command to confirm OpenSSL is installed:

```
openssl version
```

Command breakdown:

- **`openssl`**: Invokes the OpenSSL command-line tool.
- **`version`**: Displays the installed OpenSSL version.

Expected Output:
`OpenSSL 1.1.1` (or later).

Step 2: Hash a String

Hash a sample string using MD5 and SHA-256 algorithms.

MD5 Hash:

```
echo -n "Hello, OpenSSL!" | openssl dgst -md5
```

Command breakdown:

- **`echo -n "Hello, OpenSSL!"`**: Prints the string without adding a trailing newline.
- **`|`**: Pipes the output of `echo` into the OpenSSL command.
- **`openssl dgst -md5`**: Generates the MD5 hash of the input.

SHA-256 Hash:

```
echo -n "Hello, OpenSSL!" | openssl dgst -sha256
```

Command breakdown:

- **`openssl dgst -sha256`**: Generates the SHA-256 hash of the input data.

Step 3: Hash a File

To hash a file, follow the below steps:

Step A: Create a test file

Create a text file with sample content.

```
echo "This is a test file for hashing." > testfile.txt
```

Command breakdown:

- **`echo "This is a test file for hashing."`**: Outputs the string to standard output.
- **`>`**: Redirects the output to a file named `testfile.txt`.

Step B: Compute MD5 hash of the file

```
openssl dgst -md5 testfile.txt
```

Command Breakdown:

- **`openssl dgst`**: Executes the digest (hashing) function.
- **`-md5`**: Specifies the MD5 hashing algorithm.
- **`testfile.txt`**: Input file to hash.

Step C: Compute SHA-256 hash of the file

```
openssl dgst -sha256 testfile.txt
```

Command breakdown:

- **`-sha256`**: Specifies the SHA-256 hashing algorithm.

Step D: Try other algorithms

```
openssl dgst -sha1 testfile.txt
openssl dgst -sha512 testfile.txt
```

Command breakdown:

- **`-sha1`**: Uses the SHA-1 hashing algorithm.
- **`-sha512`**: Uses the SHA-512 hashing algorithm.

Step 4: Verify File Integrity

To verify file's integrity, follow the steps given below.

Step A: Modify the file

Append a new line to the file:

```
echo "Another line of text." >> testfile.txt
```

Command breakdown:

- `echo "Another line of text."`: Provides outputs for the string to standard output.
- `>>`: Appends string to the file `testfile.txt`.

Step B: Recompute the SHA-256 hash

```
openssl dgst -sha256 testfile.txt
```

Command breakdown:

- **Recomputes the SHA-256 hash** of the modified file.

Observation:
The change in the hash value indicates the file has been altered.

Step 5: Generate Hash-Based Message Authentication Code (HMAC)

Follow the below steps to generate hash-based HMAC.

Step A: Create a secret key

```
echo "supersecretkey" > secret.key
```

Command breakdown:

- `>`: Writes the string “supersecretkey” into the file `secret.key`.

Step B: Generate HMAC

```
openssl dgst -sha256 -hmac "supersecretkey" testfile.txt
```

Command breakdown:

- **`openssl dgst -sha256`**: Computes the SHA-256 hash.
- **`-hmac "supersecretkey"`**: Uses the specified key to compute the HMAC.
- **`testfile.txt`**: Input file to hash with HMAC.

Exercises

Exercise 1: Compare File Integrity with Hash Values

Objective: Verify the integrity of a file by generating its hash value and comparing it with the provided hash.

Steps Hint:

- Calculate the hash of a file.
- Compare the generated hash with a known hash value to confirm file integrity.

Hints:

1. Use the `openssl dgst` command with your preferred hash algorithm (for example, `-sha256`).
2. Ensure the hash value provided for comparison is correct and for the same hash algorithm.
3. Use a simple text file or download a sample file for testing.

Example commands:

- Generate the hash:

  ```
  openssl dgst -sha256 file.txt
  ```

- Compare the hash manually or using a script.

Exercise 2: Hash a Password and Compare Variations

Objective: Identify how small changes to input drastically change the hash output (the avalanche effect).

Steps Hint:

- Create a simple password string and hash it.
- Alter the password slightly (for example, change one character) and observe the hash output.

Hints:

1. Use `echo` to input strings directly into the hashing command.
2. Compare hashes of two similar but distinct inputs.
3. Note how output hashes are completely different, even for small input differences.

Example commands:

- Hash the original password:

```
echo -n "password123" | openssl dgst -sha256
```

- Hash the altered password:

```
echo -n "Password123" | openssl dgst -sha256
```

Exercise 3: Create and Verify a Hash-Based Authentication Token

Objective: Simulate the creation of a hash-based authentication token for securely sharing data.

Steps hint:

- Hash a combination of a secret key and a message to create an authentication token.
- Verify the token by hashing the same combination again.

Hints:

1. Concatenate the secret key and message using `echo` or manually.
2. Ensure the same hash algorithm is used for both token creation and verification.
3. Experiment with mismatched keys or messages to see how verification fails.

Example commands:

- Create a token:

```
echo -n "secretkey:message" | openssl dgst -sha256
```

- Verify the token by hashing the same input:

```
echo -n "secretkey:message" | openssl dgst -sha256
```

Conclusion

In this lab, you learned hashing using OpenSSL and explored multiple algorithms. You've also verified file integrity, and generated HMAC. These techniques are essential for ensuring data authenticity and security in cryptographic systems.

Author(s)

Dr. Manish Kumar